

Report

0.1

Customer

Epineon



Fourth Review Round

NAPQES v4 — AEAD Scheme Audit

September 2, 2026

Contents

1. Document Revisions	3
2. Introduction	4
3. System Overview	5
4. Methodology	8
5. Findings	9
5.1. Moderate	10
5.2. Minor	26
6. Conclusion	48
About Us	51
Contacts	51

1. Document Revisions

#	Date	Author	Description
0.1	September 2, 2026	Dmitry Khovratovich	Initial v4-round report. Reconciled all twenty-five third-round findings against the fourth-round source (twenty-one fixed, four partially fixed and carried, none unresolved, none regressed), walked the theorem dependency graph over all thirteen repairs and found no fix that broke a dependent result, re-derived all fifteen proofs in the document step by step with no unsupported or false step found, independently recomputed every numeric claim in the document, and filed ten findings for the v4 revision (three moderate, seven minor).

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

Client Comment

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

NAPQES (Noise-Augmented Post-Quantum Encryption Scheme) is an Authenticated Encryption with Associated Data (AEAD) scheme whose security is claimed to reduce entirely to the pseudorandomness of HMAC-SHA256, deliberately avoiding the finite-field and permutation structures used by AES-GCM and ChaCha20-Poly1305.

Version 4 keys two independently sampled secrets. The arithmetic-layer key $\mathbf{k} = (k_1, \dots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly without replacement from the set $\mathcal{P}$ of primes in $[10^6, 9.9 \times 10^6]$ (default $K = 10$), serialised as five-byte big-endian encodings; the derivation secret $\text{sk} \in \{0, 1\}^{256}$ is drawn uniformly and independently of it. sk is the only secret that ever keys an HMAC call, and $\mathbf{k}$ is used only, and un-keyed, by the arithmetic token map, so no security theorem in the paper depends on the min-entropy of $\mathbf{k}$. This round states the size of $\mathcal{P}$ exactly, as $|\mathcal{P}| = 579,947$, and recomputes from it the four figures that previously rested on an approximation: we reproduced the count by sieve and confirm it, together with $H_\infty(\mathbf{k}) = 191.46$ bits and the post-Grover figure of $2^{95.73}$ at the default K .

All internal values are produced by a domain-separated derivation

$$\text{Derive}_d(\text{sk}, N, \text{ctx}) = \text{HMAC}(\text{sk}, d \parallel N \parallel \text{ctx})$$

over nine domains covering the noise schedule, the padding codepoints, the per-position addends, the keystream and the authentication tag. The tag input is now factored into a single named quantity, $\text{TagIn}(A, B) = \text{be8}(|A|) \parallel A \parallel B$, to which every occurrence of the tag in the document refers, so the encryptor and the decryptor can no longer disagree on the width of the length prefix. The nonce is synthetic, in the manner of RFC 5297 SIV:

$$N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be8}(|A|) \parallel A \parallel M)[0:16],$$

so encryption is a pure function of the associated data and the message, and accidental nonce reuse across distinct inputs is a collision event rather than a sampling failure.

A plaintext of n codepoints is padded to a block size B given by a padding profile, newly specified in this round as a replaceable deployment parameter rather than a hard-wired rule. The normative default is the power-of-two bucket

$$B = \max(16, 2^{\lceil \log_2(n+1) \rceil}),$$

and two alternatives are given: a stride-thinned ladder and a fixed-frame profile under which every message pads to one common block size. All three take values in the same thirteen-element set, so the set of legal ciphertext lengths is profile-independent and no decryptor needs to know which profile a sender used. Padding yields $R = B + 2$ real

tokens after a two-codepoint length prefix. Each padded codepoint c becomes a token $c \cdot k + a$ for a prime $k \in \mathbf{k}$ and a derived addend $a \in [1, k - 1]$, with pseudorandom noise tokens interleaved among the real ones. Runs of consecutive noise tokens are capped at $\text{MAX_NOISE_RUN} = 19$ and the stream is then padded with filler tokens to exactly $20R$ tokens, so the token count is a deterministic function of the padding bucket alone. Tokens are serialised in fixed-width eight-byte fields, masked with a keystream derived from domain $0x07$, and authenticated by a 32-byte tag over TagIn. The ciphertext is $C = N \parallel B \parallel T$, of length $48 + 160(B + 2)$ bytes, a minimum of 2,928 bytes for any message of at most fifteen codepoints.

The message space is narrowed this round to Unicode scalar values, excluding the surrogate range, on the ground that the nonce derivation hashes the UTF-8 encoding of the message and UTF-8 is undefined on unpaired surrogates. Decryption gains two structural checks, both placed after tag verification: the recovered real-token count must equal $B + 2$ for one of the thirteen reachable block sizes, and the recovered length prefix must not exceed the padded buffer. Correctness, previously asserted in a definition environment and argued circularly, is now a proposition with a proof that begins by checking that the two tag inputs coincide.

The security section states four notions, all of them now formally defined — the previous version defined one. Confidentiality is a single-challenge, restricted-query game, and a dedicated remark states plainly that this is weaker than the deterministic authenticated-encryption notion of Rogaway and Shrimpton and that the stronger statement is not proved here. A new subsection defines a length-hiding strengthening in which the challenge messages need only share a padding bucket, proves the scheme satisfies it at the same bound, and quantifies the residual leakage at $\log_2 13 \approx 3.70$ bits per message under the default profile and at zero under the fixed-frame profile. Ciphertext integrity and chosen-ciphertext security each receive a full game definition and a complete proof; the chosen-ciphertext argument no longer invokes a composition theorem as a black box but gives its two game hops in place. A further new section separates what the arithmetic layer does and does not buy, and specifies the same construction with that layer deleted — a variant that occupies $48 + 3(B + 2)$ bytes instead of $48 + 160(B + 2)$, and for which all four theorems are shown to hold at identical bounds.

Version 4 is an expansion rather than a rewrite of the previously audited version 3: the source grows by eighty-one per cent and the count of theorem-like environments rises from twenty-two to thirty-seven, while the version-3 skeleton survives intact. Nothing of substance was removed. Three sections are entirely new, covering padding profiles, length hiding and the value of the arithmetic layer; the constant-time discussion grows from four lines to some sixty; the caveats section grows from five items to seven, adding a per-key data limit with a recommended operational cap, a timing-side-channel statement, and a candid account of a divergence between the reference implementations; and the bibliography gains the four references the previous round found missing or superseded.

/docs/

napseq-eprint-v4.tex

Scope of this round. The subject of this fourth round is the single self-contained $\text{\LaTeX}$ source napseq-eprint-v4.tex (2,281 lines). The document contains no `\input` or `\include`

directives and no ignored or commented-out normative text, so its entire content appears in the document the client distributes, and every finding below describes a defect present in it. The diff base is the previous round's `napseq-eprint-v3.tex`. No compiled PDF was submitted for this round, so locations are cited by source line interval and section heading rather than by page. No implementation artifacts were submitted either, so no specification-to-code cross-check was performed; two items remain unverified for a third round and are carried forward — the assertion that all three reference implementations enforce the two new decoder checks, whose specification half this round closes, and the unauthenticated-streaming caveat that version 3 dropped along with the streaming format it described, which version 4 neither addresses nor contradicts.

Regression outcome. All twenty-five findings of the third round were re-checked at source level. Twenty-one are fixed, four are partially fixed and carried forward as the findings below, none is unresolved and none regressed. Because no fix-response document was supplied for this round, and because the response document on file describes second-round work and already contradicted the third-round deliverable in two places, every verdict was derived from the source diff alone, judged against a reading copy from which the paper's own accounts of its corrections had been removed. A separate pass walked the theorem dependency graph over all thirteen repairs and found no fix that broke a dependent theorem, lemma or bound. All fifteen proofs in the document were reduced to their inference skeletons and checked step by step; none contains an unsupported or false step, which is a change from the previous round.

Client Comment

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the NAPQES v4 — AEAD Scheme Audit project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment

The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis

Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis

For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the NAPQES v4 — AEAD Scheme Audit codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Moderate	3	0
Minor	7	0

Fixed 0 out of 10 issues reported.

5.1. Moderate

CVF-1: open

Category: **architecture**

The length-hiding notion is parameterised by NAPQES's own padding profile, so the zero-query separation from AES-GCM and ChaCha20-Poly1305 measures the presence of that profile rather than any property of the schemes compared, and the paper never notes that the same profile applied to any AEAD achieves the same notion at the same bound

Definition 4.11 (LH-IND-CPA-det security). *Fix a padding profile $B(\cdot)$ (Section 3.6). The experiment $\text{Exp}_A^{\text{lh-ind-cpa-det}}(\lambda)$ is defined exactly as $\text{Exp}_A^{\text{ind-cpa-det}}(\lambda)$ of Definition 4.4, retaining both restrictions (i) and (ii), except that step 2's condition $|m_0| = |m_1|$ is replaced by the strictly weaker condition*

$$B(|m_0|) = B(|m_1|);$$

that is, $\mathcal{A}$ may choose challenge messages of different codepoint lengths, subject only to their falling in the same padding bucket. The advantage $\text{Adv}_{\Pi}^{\text{LH-IND-CPA-det}}(\mathcal{A})$ is defined as before.

Reference PDF p. 16 — Definition 4.11 opens by fixing a padding profile $B(\cdot)$ from Section 3.6 — the notion takes NAPQES's own length-coarsening map as its parameter.

Proposition 4.13 (Separation from AES-GCM and ChaCha20-Poly1305). *Let Σ be any AEAD scheme whose ciphertext length is a strictly increasing function of the plaintext length — in particular AES-GCM and ChaCha20-Poly1305, for which $|C| = |M| + 16$ bytes. Then there is an adversary $\mathcal{A}_{\text{len}}$ making zero oracle queries and performing a single integer comparison such that*

$$\text{Adv}_{\Sigma}^{\text{LH-IND-CPA-det}}(\mathcal{A}_{\text{len}}) = \frac{1}{2},$$

the maximum possible advantage, irrespective of the strength of the underlying primitive.

Reference PDF p. 16 — Proposition 4.13 then quantifies over any AEAD whose ciphertext length is strictly increasing in the plaintext length, and concludes the maximal advantage $\frac{1}{2}$ against AES-GCM and ChaCha20-Poly1305 in that same profile-parameterised game.

Section 4.3 is the largest new contribution in this version and, on its own terms, a good one. "Definition 4.11, LH-IND-CPA-det security" (lines 1190-1207) relaxes the confidentiality game's equal-length condition $|m_0| = |m_1|$ to the strictly weaker $B(|m_0|) =$

$B(|m_1|)$, so the challenge messages may differ in length provided they share a padding bucket; "Theorem 4.12" (lines 1209-1222) proves NAPQES satisfies it at exactly the bound of "Theorem 4.8", with no additional term. We verified that theorem step by step. Its proof asserts that the equal-length hypothesis is used in exactly one place — the appeal to "Lemma 4.6" inside the proof of "Lemma 4.10, Hiding", whose only function is to establish a shared bucket — and we traced the PRF hop, the nonce-collision union bound, the one-time-pad argument and the tag-freshness argument independently to confirm that the assertion is exactly right. The notion is genuinely stronger than "Definition 4.4", since equal lengths imply equal buckets but not conversely, and the theorem is sound.

The difficulty is in what is then built on top of it. "Definition 4.11" opens "Fix a padding profile $B(\cdot)$ (Section 3.6)", and the profiles are NAPQES constructs defined in Section 3.6. The notion is therefore not a property of a scheme; it is a property of a scheme *relative to a granularity*, and instantiating it for another scheme means importing NAPQES's own length-coarsening map into the definition of the game that the other scheme is then observed to fail. "Proposition 4.13" (lines 1243-1264) does exactly that. Its arithmetic is correct — we checked that $|m_0| = 1$ and $|m_1| = 8$ do both fall in bucket $B = 16$ under the default profile, that a scheme with $|C| = |M| + 16$ returns 17 and 24 bytes respectively, and that a single comparison therefore wins with probability 1 and advantage $\frac{1}{2}$, the maximum — but the conclusion it supports is narrower than the one drawn.

Two consequences go unstated. First, the separation is a function of the imported profile rather than of the compared schemes: under a profile with $B(n) = n + 1$ the constraint $B(|m_0|) = B(|m_1|)$ forces $|m_0| = |m_1|$ and AES-GCM satisfies the notion exactly as well as it satisfies IND-CPA, while under $\text{frame}(F)$ every pair of messages below F is admissible and AES-GCM is maximally broken — and nothing about AES-GCM has changed between those two readings. Second, and more importantly, **any** AEAD scheme achieves LH-IND-CPA-det at the same β by applying the same padding profile to its plaintext before encrypting. AES-GCM with bucket padding satisfies "Definition 4.11" at $\beta = 13$, with "Corollary 4.14"'s 3.70-bit leakage bound, at an expansion of about $1\times$ rather than $160\times$. So the comparison table's row "Length leakage per message: ≤ 3.70 bits (0 with frame)" against " $= H(n)$ " for the three competitors (line 1812) is not comparing two schemes; it is comparing a padded scheme against unpadded ones.

The paper comes remarkably close to saying this itself, which is why the omission stands out. "Remark 4.16, Coarsening, not inflation, is what hides length" (lines 1314-1333) isolates the mechanism correctly — the property is a consequence of the many-to-one map $n \mapsto B(n)$ and not of the token map, the noise schedule or the expansion factor — and "Proposition 6.1" shows the property survives deleting the arithmetic layer altogether. Both are right, and we verified both. What is missing is the last step: that the map in question is a plaintext-padding rule with no dependence on anything else in the construction, and is therefore transposable to any AEAD scheme at negligible cost. Without that step, the abstract's "we exhibit a zero-query adversary breaking the same notion outright for AES-GCM and ChaCha20-Poly1305" (lines 88-90), contribution 5 (lines 160-168), the comparison-table row and the conclusion's restatement (lines 2155-2162) all read as an absolute break of the competitors rather than as an observation about padding. That is the reading a reader is most likely to carry away, and it is the one the paper is not entitled to.

Two smaller imprecisions sit in the same section and are repaired by the same edit.

"Proposition 4.13" and "Corollary 4.14" both write $|C| = |M| + 16$ for the comparison schemes while m_0 and m_1 are codepoint sequences, so the byte length determines the codepoint count only on ASCII input; on the message space the paper actually declares, "Corollary 4.14"'s $I(n; |C|) = H(n)$ is an upper bound rather than an equality. And the measured-leakage discussion's parenthetical that the two message classes "fall in different buckets (16 and 512)" (lines 1374-1375) understates the short class: 12 to 20 codepoints straddles buckets 16 and 32. The disjointness conclusion survives, and we confirmed the 100% figure, but the parenthetical is wrong as written.

We want to be clear about what is and is not being said. We are not disputing "Theorem 4.12", "Corollary 4.14" or "Proposition 4.13"; all three are sound, and the measured table in Section 4.3 is exactly reproducible — we recomputed every digit of it, including the 2.02 and 0.94 bits of mutual information and the 10.8%, 4.6% and 1.5% recovery rates, and all agree. The finding is that a headline comparative claim is derived from a notion whose parameter is supplied by one of the two schemes being compared, and that the paper's own analysis contains everything needed to say so.

Recommendation

State the parameter dependence where the notion is defined: LH-IND-CPA-det is a property of a scheme *at a granularity* $B(\cdot)$, and a scheme's standing under it is meaningful only once the granularity is fixed.

Then reframe "Proposition 4.13" as what it proves — that a scheme whose ciphertext length determines its plaintext length satisfies the notion only at the finest granularity — and add the sentence the analysis already implies: that any AEAD scheme achieves the same notion at the same β by applying the same padding profile before encryption, so what NAPQES contributes here is the specification of a profile and a decryptor that tolerates any of them, not a property unavailable to AES-GCM.

Relabel the comparison table's length-leakage row to say that the competitors are quoted unpadding, or quote them under the same profile.

Correct $|C| = |M| + 16$ to a byte-length statement, change "Corollary 4.14"'s cross-scheme clause to an inequality, and fix the two-class parenthetical to name buckets 16 and 32 against 512.

None of this weakens the section; it makes the strongest genuine claim — that NAPQES specifies a tunable length-leakage posture down to zero, and proves what it costs — legible as the claim it is.

Client Comment

/docs/napseq-eprint-v4.tex

84 content, nonce, or associated data. Against a length-observing
adversary we
85 define and prove LH-IND-CPA-det (Theorem~\ref{thm:lh-ind-cpa}), under
which

the challenge messages may differ in length, at the same bound; we bound the residual length leakage at $\log_2 13 \approx 3.70$ bits per message, and at zero under the frame padding profile (Corollary~\ref{cor:length-leak}); and we exhibit a zero-query adversary breaking the same notion outright for AES-GCM and ChaCha20-Poly1305. We

same file

\item We define LH-IND-CPA-det (Definition~\ref{def:lh-ind-cpa-det}), the length-hiding strengthening of IND-CPA-det in which the challenge messages need only share a padding bucket, prove that NAPQES satisfies it at the same bound (Theorem~\ref{thm:lh-ind-cpa}), quantify the residual leakage over an arbitrary padding profile (Corollary~\ref{cor:length-leak}), and exhibit the zero-query adversary that achieves maximal advantage against AES-GCM and ChaCha20-Poly1305 in the same game (Proposition~\ref{prop:lh-separation}).

same file

\begin{definition}[LH-IND-CPA-det security]\label{def:lh-ind-cpa-det} Fix a padding profile $B(\cdot)$ (Section~\ref{sec:pad-profiles}). The experiment $\mathsf{Exp}^{\mathsf{LH}\text{-}\text{ind}\text{-}\text{cpa}}_{\mathcal{A}}(\lambda)$ is defined exactly as $\mathsf{Exp}^{\mathsf{ind}\text{-}\text{cpa}}_{\mathcal{A}}(\lambda)$ of Definition~\ref{def:ind-cpa-det}, retaining both restrictions (i) and (ii), except that step 2's condition $|m_0|=|m_1|$ is replaced by the strictly weaker condition $B(|m_0|) \leq B(|m_1|)$; that is, $\mathcal{A}$ may choose challenge messages of **\emph{different}** codepoint lengths, subject only to their falling in the same padding bucket. The advantage $\mathsf{Adv}^{\mathsf{LH}\text{-}\text{IND}\text{-}\text{CPA}}_{\mathcal{A}}(\lambda)$

1206 is defined as before.

1207 `\end{definition}`

same file

1243 `\begin{proposition}[Separation from AES-GCM and ChaCha20-Poly1305]`

1244 `\label{prop:lh-separation}`

1245 Let Σ be any AEAD scheme whose ciphertext length is a strictly
1246 increasing function of the plaintext length – in particular AES-GCM
and

1247 ChaCha20-Poly1305, for which $|C|=|M|+16$ bytes. Then there is an
adversary

1248 $\mathcal{A}_{\text{len}}$ making zero oracle queries and
1249 performing a single integer comparison such that

1250 `\[`

1251 $\mathbf{Adv}^{\text{LH}\text{IND}\text{CPA}}_{\Sigma}(\mathcal{A}_{\text{len}}) \leq \frac{1}{2},$
`\]`

1252 `\]`

1253 the maximum possible advantage, irrespective of the strength of the
1254 underlying primitive.

1255 `\end{proposition}`

1256

1257 `\begin{proof}`

1258 $\mathcal{A}_{\text{len}}$ outputs any (A^*, m_0, m_1) with
1259 $B(|m_0|) = B(|m_1|)$ but $|m_0| \neq |m_1|$ – for instance $|m_0|=1$ and
1260 $|m_1|=8$, both in bucket $B=16$ under the default profile. On
receiving

1261 c^* it returns $b'=0$ if $|c^*|=|m_0|+16$ and $b'=1$ otherwise,
winning

1262 with probability 1 . Restrictions (i) and (ii) are satisfied
vacuously,

1263 since no oracle query is made.

1264 `\end{proof}`

same file

1266 `\begin{corollary}[Quantified length leakage]\label{cor:length-leak}`

1267 Fix a padding profile (Section~\ref{sec:pad-profiles}) with β
1268 reachable block sizes, and let the plaintext codepoint count n be
drawn

1269 from an arbitrary distribution on the message space. Then

1270 `\[`

1271 $I\bigl(n; \cdot, |C|\bigr) \leq \log_2 \beta,$

1272 `\]`

1273 and for a sequence of m ciphertexts,

1274 $I\bigl(n_1, \dots, n_m; \cdot, |C_1|, \dots, |C_m|\bigr) \leq m \log_2 \beta,$
with no

1275 independence assumption on the n_i . Under the default
 bucket
1276 profile $\beta=13$, giving at most $\log_2 13 \approx 3.70$ bits per
message;
1277 under $\mathsf{frame}(F)$, $\beta=1$ and the leakage is exactly zero.
For
1278 AES-GCM and ChaCha20-Poly1305 on the same message space, $|C|=|M|+16$
1279 determines n exactly, so $I(n;|C|)=H(n)$ – up to 16 bits per
message.
1280 $\end{corollary}$

same file

1374 $\mathsf{coarse}(3)$ the two-class adversary succeeds with certainty,
since
1375 the two classes fall in different buckets (16 and 512) whatever
the

same file

1812 Length leakage per message ≤ 3.70 bits (0 with
 frame) $\& H(n) \& H(n) \& H(n) \setminus$

same file

2155 function of the padded-length bucket alone. Against a
length-observing
2156 adversary we prove the strictly stronger LH-IND-CPA-det notion
2157 (Theorem~\ref{thm:lh-ind-cpa}) at the same bound, bound the resulting
2158 leakage at $\log_2 13 \approx 3.70$ bits per message and zero under the
2159 frame padding profile (Corollary~\ref{cor:length-leak}),
and
2160 exhibit the zero-query adversary that breaks the same notion outright
for
2161 AES-GCM and ChaCha20-Poly1305
2162 (Proposition~\ref{prop:lh-separation}). We also show that this
property

CVF-2: open

Category: architecture

The one benefit still claimed for the arithmetic layer — extra adversary work after a nonce collision — is stated without a bound, and the paper’s own compromise caveat prices the identical search at about 2^{19} trial divisions, so a deployment is invited to pay a 53-fold bandwidth cost for a margin the document elsewhere describes as taking microseconds

This version answers the previous round's central architectural question directly and honestly. Section 6 (lines 1655-1722) states that the arithmetic layer "accounts for essentially the whole cost of this construction, and for none of its theorems", separates its value into what is proved, what is argued but not proved, and what is not claimed at all, and lists three claims it explicitly does not make. Section 6.1 (lines 1724-1792) goes further than we asked: it specifies **NAPQES-L**, the same construction with the layer deleted, proves in "Proposition 6.1" that all four security theorems hold for it with identical statements and identical bounds, tabulates the two side by side, and concludes that "the arithmetic layer costs a factor of ≈ 53 in bandwidth and buys no theorem". We verified the ciphertext-length arithmetic — $48 + 3(B + 2)$ against $48 + 160(B + 2)$, giving 102 bytes against 2,928 at the minimum bucket and a ratio tending to $160/3 = 53.3$ — and we verified "Proposition 6.1"'s four facts for NAPQES-L. "Remark 3.3" no longer cites the length-determinism lemma as a traffic-analysis justification for K , which was the specific defect recorded last round. All of that is a substantial improvement.

What remains is the single paragraph on which the whole retention decision now rests. Under "Argued, not proved: degradation under nonce collision" (lines 1681-1697), the paper says that should two distinct (A, M) pairs collide on the nonce, a construction masking padded codepoints directly would suffer "a textbook two-time pad on the plaintexts, recoverable by standard crib-dragging", whereas here the adversary obtains the exclusive-or of two token blobs in which "the noise and filler positions ... cancel to zero and are thereby revealed, while each real position yields the XOR of two affine images $c \cdot k_j + a$ and $c' \cdot k_j + a$ sharing an unknown multiplier k_j and a common addend. Recovering codepoints therefore requires additionally resolving $\mathbf{k}$." It then records this "as strictly more work for the adversary than a plain two-time pad, and as nothing more", noting that it "admits no bound we are able to state, and it must not be read as a defence".

The hedging is appropriate and we do not dispute the mechanism; we confirmed that a shared nonce produces an identical noise schedule, so the real-token positions align and the filler positions do cancel. The problem is the missing magnitude. $\mathcal{P}$ is a **public** system parameter, fixed in Section 3.1 with a normative MUST on its interval, and its size is stated exactly: $|\mathcal{P}| = 579,947 \approx 2^{19.15}$. Resolving $\mathbf{k}$ therefore means searching a public list of fewer than 2^{20} candidates per key element, testing each against the recovered affine relations. The extra work the layer imposes in the one scenario where it does anything is about 2^{19} operations — and the paper says so itself, three sections later, about the same search. The compromise caveat (lines 2066-2091) describes an adversary holding sk recovering plaintext codepoints by computing, "for each of the 579,947 candidate primes in $\mathcal{P}$ ", the corresponding addend and testing the quotient: "an $\mathcal{O}(|\mathcal{P}|)$ trial-division search that succeeds in microseconds". Both halves are in the document; they are never joined.

This matters because Section 6.1 closes by handing the reader a decision: "a deployment selecting NAPQES over NAPQES-L is selecting the beyond-model margin described above, and should do so deliberately" (lines 1790-1792). A deployment cannot select deliberately between a $53\times$ bandwidth cost and an unnamed margin. Once the margin is named — roughly 2^{19} trial divisions on top of a break that the paper agrees is "substantial plaintext leakage in both cases" — the choice becomes obvious, and the paper's own framing supports making it: the collision event is priced at $q^2/2^{128}$, the recommended operational cap of $q \leq 2^{48}$ puts it at 2^{-32} , and the margin applies only in the residual

2^{-32} case and is then worth about nineteen bits.

We are not claiming an attack, and we are not disputing that the layer makes the residue harder than a plain two-time pad. The finding is that the sole surviving justification for the dominant cost of the design is presented without the quantity that would let a reader evaluate it, when the quantity is a public parameter the paper states exactly and prices explicitly elsewhere.

Lineage: this is the residue of round-3 CVF-2, not a regression. That finding's two halves — the miscitation of the length-determinism lemma, and the absence of any comparison against the layer-free construction — are both fixed in full.

Recommendation

Quantify the margin where it is claimed. State that resolving $\mathbf{k}$ from the collision residue is a search over the public set $\mathcal{P}$, of size $|\mathcal{P}| = 579,947 \approx 2^{19.15}$ per key element, and cross-reference the compromise caveat's identical $\mathcal{O}(|\mathcal{P}|)$ trial division so that the two passages agree on the cost.

Then restate the deployment guidance with that number in it: the layer converts a two-time pad into a two-time pad plus about nineteen bits of search, in an event bounded by $q^2/2^{128}$, at a cost of $53\times$ bandwidth.

If the authors believe the residue is harder than that — for instance because the exclusive-or, unlike a difference, does not cancel the addend cleanly — then say so and give the argument, since that would be a genuinely stronger claim than the one currently made.

Either way, add the resulting sentence to the NAPQES/NAPQES-L comparison table's "Nonce-collision residue" row, which today reads "affine, $\mathbf{k}$ unknown" against "two-time pad" and so presents an unquantified advantage as a qualitative one.

Client Comment

/docs/napseq-eprint-v4.tex

```

257 | \item  $\mathcal{P}$  denotes the set of prime integers in  $[10^6,$ 
    |  $9.9 \times 10^6]$ ,
258 |     a fixed public system parameter with  $|\mathcal{P}| =$ 
    |  $579{,}947$ 
259 |     (verified by sieve; the interval is inclusive at both ends,
    | and both
260 |     endpoints are composite). Implementations MUST use this
    | interval:
261 |     a different upper bound yields a different  $\mathcal{P}$ , a
262 |     different canonical enumeration, and hence different keys.
```

same file

\paragraph{Argued, not proved: degradation under nonce collision.}
 Lemma~\ref{lem:nonce-collision} bounds the probability of two
 distinct
 \$(A,M)\$ pairs sharing a nonce by
 $\mathrm{Adv}^{\mathrm{PRF}} + 2^{-128}$, so
 what follows lies outside the model of Section~\ref{sec:security} and
 is
 \emph{not} claimed as a security property. Should such a collision
 nevertheless occur, the two ciphertexts share a keystream and the
 adversary
 obtains $B_1 \oplus B_2$. In a construction that masks padded
 codepoints
 directly, this is a textbook two-time pad on the plaintexts,
 recoverable by
 standard crib-dragging. Here it is the XOR of two token blobs: the
 noise
 and filler positions, being derived from the shared nonce alone,
 cancel to
 zero and are thereby revealed, while each real position yields the
 XOR of
 two affine images $c \cdot k_j + a$ and $c' \cdot k_j + a$ sharing an
 unknown
 multiplier k_j and a common addend. Recovering codepoints therefore
 requires additionally resolving $\mathbf{k}$. We record this as
 strictly
 more work for the adversary than a plain two-time pad, and as nothing
 more:
 it is substantial plaintext leakage in both cases, it admits no bound
 we
 are able to state, and it must not be read as a defence.

same file

\begin{table}[h]
 \centering
 \begin{tabular}{@{}lcc@{}}
 \toprule
 & NAPQES (specified) & NAPQES-L \\
 \midrule
 IND-CPA-det, INT-CTXT, IND-CCA-det & identical bounds & identical
 bounds \\
 LH-IND-CPA-det (Thm.~\ref{thm:lh-ind-cpa}) & holds & holds \\
 Length leakage (Cor.~\ref{cor:length-leak}) & $\leq \log_2 \beta$ & $\leq \log_2 \beta$ \\
 Ciphertext size, bucket B & $48 + 160(B+2)$ & $48 + 3(B+2)$ \\
 Minimum ciphertext & $2928B$ & $102B$ \\
 Secrets & $\mathbf{k}$ & sk only

```

1777 Trial-division caveat (\S\ref{sec:caveats}) & applies & vacuous \S
1778 Nonce-collision residue & affine,  $\mathbf{k}$  unknown & two-time pad
1779 \bottomrule
1780 \end{tabular}
1781 \caption{NAPQES as specified against the same construction with the
1782 arithmetic layer removed. No theorem of Section~\ref{sec:security}
1783 changes.}
1784 \label{tab:lean-comparison}
1785 \end{table}

```

same file

```

1786 NAPQES-L is stated for comparison only. It is \textbf{not} part of
1787 this
1788 specification, has no wire format, no test vectors and no reference
1789 implementation, and is not interoperable with NAPQES. We include it
1790 because
1791 the honest summary of Table~\ref{tab:lean-comparison} is that the
1792 arithmetic layer costs a factor of  $\approx 53$  in bandwidth and
1793 buys no
1794 theorem; a deployment selecting NAPQES over NAPQES-L is selecting the
1795 beyond-model margin described above, and should do so deliberately.

```

same file

```

2066 \item[Scope of  $\mathsf{sk}$  and  $\mathbf{k}$  compromise.]
2067  $\mathsf{sk}$ 
2068 is the sole cryptographic secret in this construction: every
2069 HMAC
2070 domain derivation – the noise schedule, the padding, the
2071 per-position addends, the keystream, and the tag – is keyed by
2072  $\mathsf{sk}$  alone (Section~\ref{sec:domains}).
2073 Consequently, an
2074 adversary who compromises  $\mathsf{sk}$  but not  $\mathbf{k}$ 
2075 can
2076 recover any real token's plaintext codepoint directly: the
2077 per-position addend's raw HMAC output depends only on
2078  $\mathsf{sk}$ , so for a given token the adversary can, for
2079 each of
2080 the  $579 \cdot 947$  candidate primes in  $\mathcal{P}$ , compute
2081 the corresponding addend and check whether the resulting
2082 quotient
2083 is an exact integer in a valid codepoint range – an
2084  $\mathcal{O}(|\mathcal{P}|)$  trial-division search that
2085 succeeds in
2086 microseconds.  $\mathbf{k}$ 's secrecy therefore adds
2087 \textbf{no}

```

2080 independent confidentiality margin once $\textsf{sk}$ is
 known:
 2081 compromise of $\textsf{sk}$ alone is equivalent to a full
 break,
 2082 regardless of whether $\textbf{k}$ also leaked. Operationally,
 2083 $\textbf{k}$ and $\textsf{sk}$ must therefore be protected
 with
 2084 equal rigor, as a single unit – not because splitting them
 across
 2085 separate storage or trust domains buys compartmentalized
 2086 defense-in-depth (it does not), but because $\textsf{sk}$
 2087 compromise alone is already fatal. This is consistent with,
 and
 2088 does not weaken, the security theorems of
 2089 Section~\ref{sec:security}, which assume the entire key
 material
 2090 is secret from the adversary, exactly as for any symmetric AEAD
 2091 scheme (a leaked AES-256 key is likewise a full break of
 AES-GCM).

CVF-3: openCategory: **behavior**

The C reference implementation realises a different message space from the specification and from the other two ports, so a ciphertext produced by one and decrypted by another authenticates successfully and returns a different plaintext, and the paper's remedy is to narrow the interoperability claim to ASCII rather than to make the encoding normative or the divergence detectable

- Message space $\mathcal{M}$: finite sequences of Unicode scalar values — that is, codepoints in $[\text{U} + 0000, \text{U} + 10FFFF] \setminus [\text{U} + \text{D800}, \text{U} + \text{DFFF}]$ — of length at most $\text{MAX_PLAINTEXT_CODEPOINTS} = 2^{16} - 1$, the limit imposed by the 2-codepoint big-endian length prefix written during padding (Section 3.5). The surrogate range is excluded because the synthetic-nonce derivation (Definition 3.8) hashes the UTF-8 encoding of M , and UTF-8 is undefined on unpaired surrogates; $\mathcal{M}$ is thus exactly the set of strings that admit a UTF-8 encoding. Nothing weaker is needed and nothing stronger is assumed: the arithmetic layer itself operates on the integer codepoint values and is indifferent to which subset of $[0, 2^{21})$ they range over.

Reference PDF p. 10 — Definition 3.10 declares the message space to be Unicode scalar values, with the surrogate range excluded because the nonce derivation hashes a UTF-8 encoding.

Remark 3.12 (Implementation domains). *The Python and Rust reference implementations realise $\mathcal{M}$ as specified: Rust’s `Ustr` excludes surrogates by construction, and the Python port rejects them explicitly. The C port (`C/napqes.c`) takes a NUL-terminated `char*` and maps each byte to one codepoint, so it realises $\mathcal{M}$ faithfully only for ASCII input; on non-ASCII input it produces a different, self-consistent ciphertext that the other two ports will not reproduce. This is an implementation limitation of that port, not a property of the scheme, and it is recorded in Section 12.*

Reference PDF p. 11 — Remark 3.12 then records that the C port maps each byte of a NUL-terminated string to one codepoint, so it realises that space faithfully for ASCII input only and produces ciphertext the other two ports will not reproduce.

$B_1 \oplus B_2$. In a construction that masks padded codepoints directly, this is a textbook two-time pad on the plaintexts, recoverable by standard crib-dragging. Here it is the XOR of two token blobs: the noise and filler positions, being derived from the shared nonce alone, cancel to zero and are thereby revealed, while each real position yields the XOR of two affine images $c \cdot k_j + a$ and $c' \cdot k_j + a$ sharing an unknown multiplier k_j and a common addend. Recovering codepoints therefore requires additionally resolving $\mathbf{k}$. We record this as strictly more work for the adversary than a plain two-time pad, and as nothing more: it is substantial plaintext leakage in both cases, it admits no bound we are able to state, and it must not be read as a defence.

Reference PDF p. 21 — Section 6 offers the nonce-collision residue as the arithmetic layer’s remaining benefit, conceding that it “admits no bound we are able to state”.

HMAC domain derivation — the noise schedule, the padding, the per-position addends, the keystream, and the tag — is keyed by $\mathbf{sk}$ alone (Section 3.3). Consequently, an adversary who compromises $\mathbf{sk}$ but not $\mathbf{k}$ can recover any real token’s plaintext codepoint directly: the per-position addend’s raw HMAC output depends only on $\mathbf{sk}$, so for a given token the adversary can, for each of the 579,947 candidate primes in $\mathcal{P}$, compute the corresponding addend and check whether the resulting quotient is an exact integer in a valid codepoint range — an $\mathcal{O}(|\mathcal{P}|)$ trial-division search that succeeds in microseconds. $\mathbf{k}$ ’s secrecy therefore adds **no** independent confidentiality margin once $\mathbf{sk}$ is known: compromise of $\mathbf{sk}$ alone is

Reference PDF p. 26 — Section 12 prices the identical search: an $\mathcal{O}(|\mathcal{P}|)$ trial division over the 579,947 candidate primes, which “succeeds in microseconds”.

The previous round found that the message space was defined in Unicode codepoints while the nonce was computed over UTF-8 bytes and the reference interface took bytes, with no mapping specified. Two thirds of that is now fixed. “Definition 3.10, NAPQES AEAD algorithm triple” narrows the message space to Unicode **scalar values**, excluding the surrogate range, on the correct ground that the nonce derivation hashes the UTF-8 encoding of M and UTF-8 is undefined on unpaired surrogates (lines 720-733) — so encryption is no longer undefined on part of its own declared domain. New “Remark 3.11,

The token field is wide enough for the whole of $\mathcal{M}$ (lines 746-759) discharges the notation section's outstanding " x is assumed to fit" obligation with a bound we recomputed exactly. What is left is the byte-interface question, and v4 resolves it by documentation rather than by specification.

"Remark 3.12, Implementation domains" (lines 762-772) records that Rust's `&str` excludes surrogates by construction and the Python port rejects them explicitly, but that the C port "takes a NUL-terminated `char *` and maps each *byte* to one codepoint, so it realises $\mathcal{M}$ faithfully only for ASCII input; on non-ASCII input it produces a different, self-consistent ciphertext that the other two ports will not reproduce." The caveat at lines 2093-2106 repeats this, adds that the C port "cannot represent an embedded NUL at all", states that "cross-implementation interoperability is therefore claimed for ASCII plaintexts only", and closes: "Fixing the C port to be codepoint-oriented would change no wire format but is not done here."

The candour is welcome, and the disclosure is prominent — it appears twice. But the consequence is worse than the wording suggests, and the paper does not draw it out. The two encodings do not produce mutually unreadable ciphertexts; they produce mutually readable ones with different contents. Consider the single character `é`, whose UTF-8 encoding is the two bytes `0xC3 0xA9`. The Rust or Python port, handed that string, encrypts one codepoint `U+00E9`. The C port, handed the same bytes, encrypts two codepoints `U+00C3` and `U+00A9`. Both are valid scalar values, both are inside $\mathcal{M}$, and the C port's output is a perfectly well-formed NAPQES ciphertext under the same key. Hand that ciphertext to the Rust port and every check passes: the length is a legal $48 + 160(B + 2)$, the tag verifies because the tag is computed over the masked blob and the associated data and knows nothing about how the caller obtained its codepoints, the bucket check at step (6) passes, the length-prefix check at step (8) passes, and step (9) returns two codepoints. The receiving application is handed the string `Ã©`, authenticated, with no error and no indication that anything was lost. Authenticated encryption that returns a different plaintext than was encrypted, silently, is a stronger failure than an interoperability gap; it is the failure mode that authentication exists to prevent, arriving through the encoding layer instead of the cryptographic one.

Nothing in the wire format lets a decryptor detect which convention produced a ciphertext. There is no version byte, no encoding tag, no domain separator distinguishing byte-mode from codepoint-mode, and the message space itself cannot distinguish them because both interpretations land inside it. The one mechanism the paper offers as evidence of cross-implementation agreement — the known-answer corpus, which conforming implementations "must produce byte-identical ciphertext for" (lines 1992-1993) — is by the paper's own statement confined to "the domain the KAT corpus of Section 10 covers", namely ASCII (lines 2104-2105). So the corpus is structurally incapable of detecting this class of divergence, and a deployment that validates all twenty vectors on all three ports learns nothing about the case that breaks.

We note that the paper is entitled to declare the C port non-conforming, and in substance it does. But the C port is shipped in the published distribution, named as one of three reference implementations in Section 8, cited in Section 8.4's constant-time discussion, and offered as one of the three ports the corpus establishes agreement between. A reader who takes the paper's own framing — "implementation notes across Python, Rust, and C reference implementations" — and deploys the C port for non-ASCII traffic gets

silent plaintext corruption across a mixed-language deployment. That is a deployment hazard rather than a documentation defect, which is why we have not filed it as one.

Recommendation

Make the encoding normative and make a mismatch impossible rather than merely disclosed.

The cleanest route is the one the previous round already suggested and which this version's own analysis now makes cheap: define the message space as **byte strings**, specify that each plaintext byte becomes a codepoint in $[0, 255]$, and state the nonce derivation over the same byte string. That makes the C port's behaviour the specification, removes the surrogate question entirely, lets the length cap be stated in the unit the interface enforces, and — by "Remark 3.11"'s own bound, since $255 < 0x10FFFF$ — cannot overflow the token field. The Rust and Python ports would then decode nothing and pass bytes straight through, which is a smaller change than fixing the C port.

If instead the codepoint domain is retained, then fix the C port, and until it is fixed do not present it as a reference implementation: state that it implements a distinct, non-interoperable encoding, and add a domain-separated encoding tag to the tag input so that a ciphertext produced under one convention is *rejected* rather than silently mis-decrypted under the other.

In either case add known-answer vectors containing non-ASCII plaintext, so that the corpus can detect the divergence it is currently blind to, and say explicitly in Section 10 that the current corpus cannot.

Client Comment

/docs/napseq-eprint-v4.tex

```

720 | \item Message space  $\mathcal{M}$ : finite sequences of Unicode
721 |       \emph{scalar values} – that is, codepoints in
722 |        $[\mathtt{U+0000}, \mathtt{U+10FFFF}] \setminus [\mathtt{U+D800}, \mathtt{U+DFFF}]$ 
723 |       – of length at most
724 |        $\mathrm{MAX\_PLAINTEXT\_CODEPOINTS} = 2^{16} - 1$ , the
725 |       limit imposed by the 2-codepoint big-endian length prefix
written
726 |       during padding (Section~\ref{sec:padding}). The surrogate
range is
727 |       excluded because the synthetic-nonce derivation
728 |       (Definition~\ref{def:synthnonce}) hashes the UTF-8 encoding
of  $M$ ,
729 |       and UTF-8 is undefined on unpaired surrogates;  $\mathcal{M}$ 
is thus

```

exactly the set of strings that admit a UTF-8 encoding. Nothing weaker is needed and nothing stronger is assumed: the arithmetic layer itself operates on the integer codepoint values and is indifferent to which subset of $[0, 2^{21})$ they range over.

same file

```
\begin{remark}[Implementation domains]\label{rem:impl-domain}
The Python and Rust reference implementations realise  $\mathcal{M}$ 
as
specified: Rust's \&str excludes surrogates by construction,
and
the Python port rejects them explicitly. The C port
(\&C/napqes.c) takes a NUL-terminated \&char\,* and
maps each
\&byte to one codepoint, so it realises  $\mathcal{M}$  faithfully
only
for ASCII input; on non-ASCII input it produces a different,
self-consistent
ciphertext that the other two ports will not reproduce. This is an
implementation limitation of that port, not a property of the scheme,
and
it is recorded in Section~\ref{sec:caveats}.
\end{remark}
```

same file

```
\subsection{C Implementation}
A C port (\&C/napqes.c) mirrors the same construction
(\&napqes\_encrypt\_bytes\_v8,
\&napqes\_decrypt\_bytes\_v8).
```

same file

The corpus for the scheme specified in this paper is `\&tests/kat/v8\_vectors.json`. It contains 20 vectors: 12 positive (ids `\&W001`--`\&W012`) covering the empty message, a single character, the block boundaries at 15, 16 and 32 codepoints, 1-element and 10-element prime tuples $\mathbf{k}$, empty, binary and 64-byte associated data (the last pinning the 8-byte $\mathrm{be8}$ AAD length prefix of Definition~\ref{def:taginput}), and mixed-case punctuation; and 8 negative. Of the negative vectors, `\&W-N01`--`\&W-N05` exercise

1981 authentication (flipped tag byte, flipped masked-blob byte, wrong
 1982 A ,
 1983 wrong sk) and the 48-byte parse floor, while
 1984 W-N06 and W-N08 are **validly tagged** but
 1985 structurally
 1986 malformed and therefore reach the post-authentication checks of
 1987 Dec step (3): a token count that is not a multiple of
 1988 $\text{MAX_NOISE_RUN}+1$; a real-token count that is such a
 1989 multiple but
 1990 is not $B+2$ for any reachable bucket B ; and a padded length prefix
 1991 claiming more codepoints than the padded buffer holds. Each positive
 1992 vector
 1993 fixes $(\mathbf{k}, \text{sk}, A, M)$ and the expected
 1994 ciphertext; because Enc is a pure function of its inputs
 1995 (Definition~[def:aead-triple](#)), no nonce field appears and none
 1996 is
 1997 needed. Conforming implementations must produce byte-identical
 ciphertext
 for every positive vector and reject every negative vector. The
 generator
[tests/gen_kats_v8.py](#) regenerates the file and verifies
 parity on
 any port; [tests/test_kats.py](#) and
[rust/src/kat_cross_check.rs](#) consume it from Python and
 Rust
 respectively.

same file

2093 **Message domain and the C port.** The message space
 2094 M
 2095 excludes the surrogate range
 2096 $[\text{U+D800}, \text{U+DFFF}]$,
 2097 since the synthetic-nonce derivation hashes the UTF-8
 2098 encoding of
 2099 M and UTF-8 is undefined there. Rust enforces this by type
 2100 and
 2101 Python rejects surrogates explicitly. The C port
 2102 ([C/napqes.c](#)) instead maps each **byte** of a
 2103 NUL-terminated char to one codepoint
 (Remark~[rem:impl-domain](#)), so it agrees with the other
 two
 ports on ASCII input only; on non-ASCII input it produces a
 different, self-consistent ciphertext, and it cannot
 represent an
 embedded NUL at all. Cross-implementation interoperability is

```

2104 |         therefore claimed for ASCII plaintexts only, which is the
domain the
2105 |         KAT corpus of Section~\ref{sec:kats} covers. Fixing the C
port to be
2106 |         codepoint-oriented would change no wire format but is not
done here.

```

5.2. Minor

CVF-4: open

Category: **algorithm**

The comparison table states ciphertext expansion as a fixed 160-fold against a 1-fold baseline, but 160 is bytes per *padded* codepoint; relative to the plaintext the expansion ranges from 160-fold to 343-fold and is 2,928-fold for any message of at most fifteen codepoints, as the paper's own minimum-ciphertext figure shows

The previous round found this figure stated as $20\times$ when the on-wire cost was eight times that, and the correction has been applied thoroughly in the body. Section 7 now reads "a hard, deterministic 160 bytes per padded codepoint" (line 1830), the caveat reads "Fixed at exactly 20 tokens, i.e. 160 bytes, per padded codepoint" (lines 2037-2038), the conclusion reads "the fixed 160-byte-per-codepoint ciphertext expansion" (line 2176), and the closed form $|C| = 48 + 160 (B(|M|) + 2)$ appears twice (lines 678-680, 738-743). Every one of those is correct and we recomputed all of them. The units are stated in each case, and "per padded codepoint" is the right unit for what is being described.

The comparison table is the exception. Its expansion row reads " $160\times$ (fixed)" for NAPQES against " $1\times$ " for AES-GCM, " $1\times$ " for ChaCha20-Poly1305 and " $1\times$ " for Ascon (line 1811). Those neighbouring entries fix the unit unambiguously as a plaintext-relative ratio — AES-GCM's ciphertext is $|M| + 16$ bytes, which is $1\times$ the plaintext plus a constant — and against that baseline $160\times$ is not what NAPQES achieves. Substituting the paper's own closed form and taking one byte per codepoint, the ratio $|C|/n$ is 2,928 at $n = 1$, 195.2 at $n = 15$, **343.0** at $n = 16$, 177.0 at $n = 31$, 164.2 at $n = 1,000$ and 160.01 at the maximum $n = 65,535$. The figure is $160\times$ only asymptotically; because B rounds up to a power of two the ratio oscillates, peaking at $343\times$ just past each bucket boundary and settling toward $320\times$ for large buckets, and below sixteen codepoints it is unboundedly larger because every message pays the 2,928-byte floor. The paper computes that floor itself, three sections earlier (lines 674-680), so the document contains the refutation of its own table cell for the second round running —

last time by a factor of eight, this time by a factor of two and by the loss of the "(fixed)" qualifier's truth.

Nothing downstream depends on it: the trade-off discussion that follows the table now correctly declines to offer the expansion as a traffic-analysis countermeasure, "Proposition 4.15, Expansion is length-neutral" proves the expansion carries no length confidentiality whatever, and Section 6.1 quantifies the cost against NAPQES-L using ciphertext lengths rather than ratios. This is a table-cell accuracy finding, and the severity is low. It is worth fixing because the table is the paper's most-read comparative artifact, because "(fixed)" asserts precisely the property the number does not have, and because a reader budgeting bandwidth for short messages — the case the design is worst at, and the case a control-plane or telemetry deployment will have — will be out by a factor of eighteen.

Recommendation

Give the row the unit it is being compared in, or change the comparison. The accurate entry is a range: $160\times$ to $343\times$ of the plaintext for messages of sixteen codepoints or more, and a flat 2,928 bytes for anything shorter.

If a single number is wanted, state it as an additive-plus-multiplicative form — $48 + 160(B(|M|) + 2)$ bytes, alongside AES-GCM's $|M| + 16$ — which is exact for both columns and makes the short-message floor visible.

Keep the body text's "per padded codepoint" phrasing, which is correct, and add a clause distinguishing it from the plaintext-relative figure so the two cannot be read as the same number.

Since the row is the only place the word "(fixed)" appears about expansion, either drop it or qualify it as fixed *per padded codepoint*.

Client Comment

/docs/napseq-eprint-v4.tex

```

674 true minimum length of a well-formed ciphertext is larger, since every
675 message – including the empty string – pads to at least  $B=16$ 
codepoints
676 ( $R=18$  real tokens), giving
677  $N_{\{\mathrm{tok}\}} = 18 \cdot 20 = 360$ ,  $|B|=2880$  bytes, and
678  $|C| = 16+2880+32 = 2928$  bytes at minimum. In general
679  $|C| = 48 + 160 \cdot (B(|M|)+2)$  bytes, a closed form in the padding
bucket
680 alone.

same file

738 \[
739 |C| \ ;=\; 48 + 160\bigl(B(|M|)+2\bigr)\ \text{bytes};
740 \]
```

741 | the parse-time
 742 | rejection floor is 48 bytes, but the true minimum length of a
 743 | well-formed ciphertext is $48 + 160 \cdot 18 = 2928$ bytes.

same file

1811 | Ciphertext expansion & $160 \times$ (fixed) & $1 \times$ & $1 \times$ &
 | $1 \times$ \\
 same file

2037 | \item[Ciphertext expansion.] Fixed at exactly 20 tokens, i.e.\
 | 160
 2038 | bytes, per padded codepoint (Section~\ref{sec:comparison}).
 The
 2039 | expansion is deterministic rather than message-dependent,
 which is
 2040 | what Lemma~\ref{lem:length-det} requires, but it is
 | \emph{not} a
 2041 | traffic-analysis countermeasure: by
 2042 | Proposition~\ref{prop:expansion-neutral} it contributes zero
 bits
 2043 | of length confidentiality. It is a cost of the arithmetic
 layer,
 2044 | whose retention is argued and bounded in
 2045 | Section~\ref{sec:arith-layer-value} and quantified against
 the same
 2046 | construction without that layer in
 Section~\ref{sec:lean-profile}.

same file

2176 | the fixed 160 -byte-per-codepoint ciphertext expansion of the
 2177 | noise-injection design, which buys no theorem and is retained for the
 2178 | beyond-model reasons set out in Section~\ref{sec:arith-layer-value},
 the

CVF-5: openCategory: **documentation**

The caveats section recommends the fixed-frame padding profile to deployments needing zero length leakage without repeating that it restricts the message space, that it charges every message the largest ciphertext, or that a receiver has no way to tell whether a sender used it

Section 3.6 introduces the padding profile as a replaceable parameter and specifies three of them. The definitions are careful and internally consistent, and we verified the claim

that all three take values in the same thirteen-element block-size set, so a decryptor needs no knowledge of the sender's profile and any well-formedness check on the recovered real-token count remains a check against the same fixed set. "Corollary 4.14, Quantified length leakage" is then stated over an arbitrary profile, giving leakage at most $\log_2 \beta$ bits for β reachable block sizes — about 3.70 bits under the default and exactly zero under $\text{frame}(F)$. Both are correct.

Section 3.6 also states the two conditions that make $\text{frame}(F)$ usable: it is defined "for a fixed $F \in \{2^4, \dots, 2^{16}\}$, on the restricted message space $n < F$ " (lines 491-494), and its zero leakage comes "at the cost of every message paying the F -sized ciphertext" (lines 510-512). Neither condition survives into the places where the profile is actually recommended. The caveat on padding-bucket leakage tells deployments "requiring zero length leakage" to "select $\text{frame}(F)$ " (Section 3.6), which needs no change to the wire format and no cooperation from the decryptor; no external fixed-frame transport is required" (lines 2059-2064). The abstract offers "at zero under the frame padding profile" (lines 87-89), the comparison table offers "0 with frame" (line 1812), and the conclusion repeats it (lines 2158-2159). In none of those four places is the message-space restriction or the bandwidth consequence mentioned, and the consequence is severe: at the only value of F that covers the whole declared message space, $F = 2^{16}$, every message — including the empty string — costs $48 + 160 \cdot 65,538 = 10,486,128$ bytes. A reader who takes the caveat at face value and selects $\text{frame}(F)$ for a deployment sending messages of mixed length either discovers that messages of $n \geq F$ are now rejected, or pays ten megabytes per message.

The second gap is structural rather than editorial, and it follows from the property the paper presents as a convenience. The profile is "never transmitted and does not appear in the wire format" (lines 505-506), and "any conforming decryptor accepts any profile's output unchanged" (lines 502-503). That makes the profile the only security-relevant parameter in the construction whose mismatch is undetectable. A wrong $\mathbf{k}$, a wrong K or a wrong sk all cause decryption to fail; a sender that silently falls back from $\text{frame}(65536)$ to the default bucket profile — through a configuration error, a library default, a downgrade, or a build that predates the profile parameter — produces ciphertexts that every receiver accepts, and the receiver has no signal at all. Since the profile is what "Corollary 4.14"'s β is, a deployment believing it has zero length leakage has no mechanism to confirm it, and the leakage it actually gets is whatever the sender chose. The paper presents "no cooperation from the decryptor" as the feature that makes profiles cheap, and it is; but it is simultaneously the reason the guarantee is unenforceable, and only the first half is stated.

The security consequence is bounded — the worst case is the default profile's 3.70 bits, which is the posture the paper analyses anyway — so the severity is low. It is worth fixing because the zero-leakage claim now appears in the abstract, a table row and the conclusion, and because a guarantee that cannot be verified by the party relying on it should be described as a deployment convention rather than as a property of the scheme.

Recommendation

Carry the two preconditions to every site where the profile is recommended. The caveat, the abstract, the table row and the conclusion should each say that $\text{frame}(F)$ gives zero length leakage *on the message space $n < F$ and at a uniform ciphertext*

cost of $48 + 160(F + 2)$ bytes for every message, with the $F = 2^{16}$ figure spelled out once so the trade is visible.

Then state the enforcement gap plainly, in the caveats section: because the profile is neither transmitted nor derivable from the ciphertext, a receiver cannot determine which profile a sender used, and a silent downgrade to a finer profile is undetectable; deployments relying on a leakage bound must enforce the profile as a configuration invariant on the sending side, and should treat it as they would any other out-of-band key parameter.

If detection is wanted, the natural mechanism is to bind the profile identifier into the tag input alongside the associated-data length, which costs one byte of domain-separated context and makes a mismatch a rejection rather than a silent difference; that is worth considering explicitly, since the paper currently frames the absence of any such binding as an advantage.

Client Comment

/docs/napseq-eprint-v4.tex

```
87 | the residual length leakage at  $\log_2 13 \approx 3.70$  bits per
    | message, and
88 | at zero under the  $\mathsf{frame}$  padding profile
89 | (Corollary~\ref{cor:length-leak}); and we exhibit a zero-query
    | adversary
```

same file

```
491 | \item[\mathsf{frame}(F).]  $B(n)=F$  for a fixed
492 |  $F \in \{2^4, \ldots, 2^{16}\}$ , on the restricted message space
    |  $n < F$ :
493 | every message pads to one common block size, so exactly one
    | value is
494 | reachable.
```

same file

```
505 | agreed out of band, exactly like  $\mathbf{k}$  and  $K$ ; it is never
506 | transmitted and does not appear in the wire format.
507 |
508 | Corollary~\ref{cor:length-leak} is stated over an arbitrary profile:
    | length
509 | leakage is at most  $\log_2$ (number of reachable values) bits – about
    |  $3.70$ 
510 | bits under  $\mathsf{bucket}$ , and exactly zero under
511 |  $\mathsf{frame}(F)$ , at the cost of every message paying the  $F$ -sized
```

```

512 ciphertext. Deployments whose threat model is dominated by traffic
analysis
513 should select  $\mathsf{frame}(F)$  for the largest  $F$  their bandwidth
budget
514 tolerates; this, and not the expansion factor, is the knob that trades
515 bandwidth for length confidentiality
516 (Remark~\ref{rem:coarsening-not-inflation}).

same file

1812 Length leakage per message  $\leq 3.70$  bits ( $0$  with
 $\mathsf{frame}$ )  $\& \mathcal{H}(n) \& \mathcal{H}(n) \& \mathcal{H}(n) \setminus \setminus$ 

same file

2059 the bucket itself. Deployments requiring strictly less than
this
2060 should select a coarser padding profile, and those requiring
zero
2061 length leakage should select  $\mathsf{frame}(F)$ 
2062 (Section~\ref{sec:pad-profiles}), which needs no change to
the wire
2063 format and no cooperation from the decryptor; no external
2064 fixed-frame transport is required.

same file

2158 leakage at  $\log_2 13 \approx 3.70$  bits per message and zero under the
2159  $\mathsf{frame}$  padding profile (Corollary~\ref{cor:length-leak}),
and

```

CVF-6: openCategory: **readability**

The first contribution still writes the synthetic nonce without its sixteen-byte truncation, so the front matter describes a thirty-two-byte nonce that the construction does not use

The previous round found three mismatches between the front matter and the body: the nonce formula given without truncation in both the abstract and the first contribution, scrambled cross-references in the third contribution, and an abstract promise of performance measurements the paper does not report. Two are fixed and the third is half fixed.

The cross-references are repaired: the third contribution now reads “We prove IND-CPA-det (Definition 4.4, Theorem 4.8), INT-CTXT (Definition 4.17, Theorem 4.18), and IND-CCA-det (Definition 4.19, Theorem 4.20) security” (lines 145-149), so each claimed result points at both the definition that states the notion and the theorem that proves it, and the

dangling trailing reference is gone. The performance claim is withdrawn: the abstract now says only "We discuss the post-quantum security level and implementation notes across Python, Rust, and C reference implementations" (lines 94-96), which no longer contradicts the conclusion's listing of a performance evaluation as future work. Both repairs are complete.

The nonce formula was corrected in one of its two places. The abstract now reads

$$N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be8}(|A|) \parallel A \parallel M)[0:16]$$

(line 70), matching "Definition 3.8, Synthetic nonce". The first contribution, twenty lines later, still reads $N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be8}(|A|) \parallel A \parallel M)$ with no truncation (line 137). As written it describes a thirty-two-byte nonce, which is not the construction: the normative definition takes the first sixteen bytes (lines 630-637), the wire format carries a sixteen-byte nonce field (lines 669-672), the parse-time floor of forty-eight bytes is computed from it, and "Lemma 4.7, Nonce collision"'s bound of 2^{-128} depends on the truncation length.

This is the same defect at the same pair of sites that the previous round reported about the missing $\text{be8}(|A|)$ length prefix: that repair also reached one site before the other, and both sites were eventually brought into line. The pattern is worth noting to the authors — an abbreviated formula that appears twice will keep drifting until the two copies are the same object.

Nothing depends on it and no reader who reaches Section 3.8 will be misled, so the severity is low. It is worth fixing because the contributions list is the second thing read after the abstract and is where a reader forms their model of the construction, and because the two statements of the same formula now disagree with each other within thirty lines.

Recommendation

Add $[0:16]$ to the first contribution's formula so that it matches the abstract and "Definition 3.8".

Better, remove the duplication that causes the drift: state the nonce once, as a displayed named quantity in Section 3.8, and have the abstract and the contributions refer to it by name and reference rather than restating the expression — the same device "Definition 3.4, Tag input" uses for the tag input, which is what made this round's much more serious 4-byte versus 8-byte length-prefix divergence impossible to reintroduce.

Client Comment

/docs/napseq-eprint-v4.tex

70 $N = \text{HMAC}(\text{sk}, 0x0A \parallel \text{be8}(|A|) \parallel A \parallel M)[0:16]$,
same file

```

133 | \item We fully specify NAPQES (Section~\ref{sec:construction}): an
134 |         independent HMAC-domain-derivation secret  $\textsf{sk}$  and
135 |         arithmetic-layer key  $\textbf{k}$ , sampled independently,
together
136 |         with a synthetic, message-bound nonce
137 |
 $N = \text{HMAC}(\textsf{sk}, \texttt{0x0A} \parallel \text{be8}(|A|) \parallel A \parallel M)$ 
138 |         (Definition~\ref{def:synthnonce}) and a fixed-width,
per-bucket-ceiling
139 |         wire format (Section~\ref{sec:wire-format}) under which
ciphertext
140 |         length depends only on the padded-length bucket.

same file

630 | \begin{definition}[Synthetic nonce]\label{def:synthnonce}
631 | For  $\textsf{sk} \in \{0,1\}^{256}$ ,  $A \in \{0,1\}^*$ ,
 $M \in \mathcal{M}$ 
632 | (encoded as UTF-8 bytes),
633 | \[
634 |   N(\textsf{sk}, A, M) \stackrel{\text{def}}{=} \text{HMAC}(\textsf{sk}, \texttt{0x0A} \parallel \text{be8}(|A|) \parallel A \parallel M)
635 |   \text{be8}(|A|) \parallel A \parallel M \text{bigr})[0{:}16].
636 | \]
637 | \end{definition}

same file

669 | \[
670 |   C = N \parallel \widetilde{B} \parallel T,
671 | \]
672 | a 16-byte nonce, the masked token blob, and the 32-byte tag. A
decryptor

```

CVF-7: openCategory: **documentation**

Three defects in normative text: the decryption algorithm writes the addend as a raw digest rather than as the reduced value it defines, the test-vector description attributes the two new post-authentication checks to the pre-authentication step, and two numbered remarks are nested inside the algorithm-triple definition so that it is interrupted before the algorithms are given

Three separate places where the specification's own text does not say what the specification means. All three are residues of otherwise complete repairs made this round, and all three are in normative or near-normative material.

The addend in decryption. "Definition 3.6, Token-construction primitives" specifies the real-token addend as

$$\text{real_addend}(\text{real_idx}, k) = (\text{Derive}_{0x01}(\text{sk}, N, \text{be5}(\text{real_idx})) [0:4]_{\text{be32}} \bmod (k-1)) + 1$$

, a value in $[1, k-1]$ (lines 525-533). Step (7) of Dec recovers the padded codepoint as $(t - a)/k_{(\text{real_idx} \bmod K)+1}$ "where a is the real_idx -th addend $\text{Derive}_{0x01}(\text{sk}, N, \text{real_idx})$ " (lines 807-812). That expression is the raw thirty-two-byte HMAC output: it drops the $\text{be5}(\cdot)$ encoding of the counter, the $[0:4]_{\text{be32}}$ truncation, the reduction modulo $k-1$ and the $+1$. Read literally, a is a digest rather than an integer in $[1, k-1]$ and the subtraction is ill-typed. The intent is unambiguous from "Definition 3.6", and "Proposition 3.14, Correctness"'s proof supplies the reading in words ("exactly the prime and addend Enc used at that index"), so nothing is actually wrong downstream — but this is the one step of the specification a reader must implement from scratch, in the algorithm the paper describes as normative, and it is the identical failure mode as this round's 4-byte versus 8-byte length-prefix repair: the same quantity written two ways in two algorithms. The step is also where the previous round asked for the prime selection and the addend index to be named, which was done; the reduction was not carried across at the same time.

The test vectors' cited step. "Remark 3.13, The post-authentication structural checks" establishes the ordering that makes the new decoder checks safe: steps (6) and (8) run *after* the tag comparison of step (5), so they are unreachable without sk and play no part in the integrity argument (lines 819-832). Section 10 then describes the three new negative vectors as "*validly tagged* but structurally malformed and therefore reach the post-authentication checks of Dec step (3)" (lines 1983-1986). Step (3) is the *pre*-authentication divisibility check, which runs before the tag comparison; the post-authentication checks are steps (6) and (8). The description is wrong in both directions at once — it names the wrong step, and it describes as post-authentication a step that is pre-authentication. It also lumps three vectors under two checks: of the three cases listed, "a token count that is not a multiple of $\text{MAX_NOISE_RUN} + 1$ " is caught at step (3) before the tag is examined, so it does not need to be validly tagged at all, while the other two — a real-token count that is a multiple but is not $B + 2$ for any reachable bucket, and a length prefix claiming more codepoints than the buffer holds — are exactly steps (6) and (8). "Remark 3.13" itself says "vectors W-N06-W-N08 ... pin them", where "them" is the two checks. Since the whole point of "Remark 3.13" is that the ordering is normative, a description that misplaces it in the section offering the vectors as evidence is worth correcting.

The nested remarks. "Definition 3.10, NAPQES AEAD algorithm triple" opens at line 708 and closes at line 817. Inside it, after the itemised key, associated-data, message and ciphertext spaces and before the KeyGen, Enc and Dec paragraph bodies, sit two full theorem-like environments: "Remark 3.11" on the token field's width (lines 746-760) and "Remark 3.12" on the three ports' message domains (lines 762-772). The shared theorem counter advances through them, so the definition of the algorithm triple is interrupted by two numbered remarks, in the definition environment's own styling, before the algorithms it defines are stated. Both remarks are good content and both are new this round; neither belongs inside the environment. We have not been able to confirm how this renders, since no compiled PDF was submitted for this round, but the structure is visible in the source and the numbering consequence is unavoidable.

None of the three has a security consequence, which is why this is a single low finding rather than three. All three are in text a conforming implementer reads as normative.

Recommendation

Write step (7)'s addend as `real_addend(real_idx, k)`, referring to "Definition 3.6" by name, so that the reduction cannot drift from the definition — the same device "Definition 3.4, Tag input" applies to the tag input, and for the same reason.

Correct Section 10's sentence to say that W-N06 exercises the pre-authentication divisibility check at step (3) while W-N07 and W-N08 are validly tagged and reach the post-authentication checks at steps (6) and (8), and cross-reference "Remark 3.13" for the ordering.

Move "Remark 3.11" and "Remark 3.12" out of "Definition 3.10" — "Remark 3.11" naturally follows the wire-format section where the eight-byte token field is introduced, and "Remark 3.12" belongs beside the implementation section or the caveat it duplicates — so that the definition runs uninterrupted from its typed spaces to its three algorithms.

Client Comment

/docs/napseq-eprint-v4.tex

```

525 \begin{definition}[Token-construction
primitives]\label{def:token-prims}
526 For a key element  $k$ , position counters  $ct\_pos$ ,  $real\_idx$ :
527 \begin{align*}
528   \texttt{is\_noise}(ct\_pos) \mathrel{=}&
\mathbf{1} \mathbf{bigl[} \backslash, \mathrm{Derive}_{\{0x00\}}(\mathsf{sk}, N,
\mathrm{be5}(ct\_pos))[0{:}8]_{\mathrm{be64}} < \theta(N) \backslash, \mathbf{bigr}] , \backslash
529   \texttt{noise\_char}(ct\_pos) \mathrel{=}&
\mathbf{bigl[} \mathrm{Derive}_{\{0x04\}}(\mathsf{sk}, N,
\mathrm{be5}(ct\_pos))[0{:}4]_{\mathrm{be32}} \bmod 96 \mathbf{bigr}] + 32 , \backslash
530   \texttt{noise\_addend}(ct\_pos, k) \mathrel{=}&
\mathbf{bigl[} \mathrm{Derive}_{\{0x05\}}(\mathsf{sk}, N,
\mathrm{be5}(ct\_pos))[0{:}4]_{\mathrm{be32}} \bmod (k-1) \mathbf{bigr}] + 1 ,
\backslash
531   \texttt{real\_addend}(real\_idx, k) \mathrel{=}&
\mathbf{bigl[} \mathrm{Derive}_{\{0x01\}}(\mathsf{sk}, N,
\mathrm{be5}(real\_idx))[0{:}4]_{\mathrm{be32}} \bmod (k-1) \mathbf{bigr}] +
1 .
532 \end{align*}
533 \end{definition}

```

same file

```

746 \begin{remark}[The token field is wide enough for the whole of
    \$\mathcal{M}\$]
747 \label{rem:token-fit}
748 Every emitted token has the form  $c \cdot k_j + a$  with  $c$  a padded
    codepoint,
749  $k_j \in \mathcal{P}$  and  $a \in [1, k_j - 1]$ 
750 (Section~\ref{sec:token-construction}), so
751  $c \cdot k_j + a \leq (c+1)k_j - 1$ . Taking the extreme values admitted by
752  $\mathcal{M}$  and  $\mathcal{P}$  —  $c = \mathtt{0x10FFFF}$  and
753  $k_j = 9\{, \}899\{, \}999$ , the largest prime in  $\mathcal{P}$  — gives a
    maximum
754 token of  $11\{, \}029\{, \}707\{, \}685\{, \}887 \approx 2^{43.33}$ . The fixed
    8-byte
755 big-endian token field therefore never overflows, with a margin of a
    factor
756 of over  $1.6 \times 10^6$ . This bound is unconditional: it holds for
    every
757  $M \in \mathcal{M}$ , every  $\mathbf{k}$ , and every noise realisation,
    so
758 Lemma~\ref{lem:length-det} and the serialisation are well-defined on
    the
759 entire message space.
760 \end{remark}
761
762 \begin{remark}[Implementation domains]\label{rem:impl-domain}
763 The Python and Rust reference implementations realise  $\mathcal{M}$ 
    as
764 specified: Rust's \texttt{\&str} excludes surrogates by construction,
    and
765 the Python port rejects them explicitly. The C port
766 (\texttt{C/napqes.c}) takes a NUL-terminated \texttt{char\,*} and
    maps each
767 \emph{byte} to one codepoint, so it realises  $\mathcal{M}$  faithfully
    only
768 for ASCII input; on non-ASCII input it produces a different,
    self-consistent
769 ciphertext that the other two ports will not reproduce. This is an
    implementation limitation of that port, not a property of the scheme,
    and
770 it is recorded in Section~\ref{sec:caveats}.
771 \end{remark}
772
    same file

807 token  $t$  and recover the padded codepoint as
808  $(t-a)/k_{\{(\mathrm{real\_idx} \bmod K)+1\}}$ , where  $a$  is the
809  $\mathrm{real\_idx}$ -th addend

```

```

810  $\mathrm{Derive}_{\{0x01\}}(\mathsf{sk}, N, \mathrm{real\_idx})$  and
811  $\mathrm{real\_idx}$  counts real tokens from  $0$  – exactly the prime
and
812 addend  $\mathsf{Enc}$  used at that index; stop once  $R$  real tokens
have
same file
819  $\begin{remark}$ [The post-authentication structural checks]
820  $\label{rem:dec-structural}$ 
821 Steps (6) and (8) are performed  $\emph{after}$  the tag comparison of
step (5),
822 so they are never reachable on an input an adversary without
 $\mathsf{sk}$ 
823 could construct, and they play no part in the INT-CTXT argument. They
are
824 specified nonetheless because step (7) is not total without them:  $R$ 
is
825 recovered from  $|C|$  and  $n$  from the decrypted buffer, so a
malformed but
826 correctly tagged ciphertext – which a key holder can trivially
produce –
827 would otherwise drive an out-of-range read or an allocation sized by
a field
828 the caller did not choose. Divisibility by
 $\mathrm{MAX\_NOISE\_RUN}+1$ 
829 alone does not imply step (6):  $R=20$  passes step (3) and fails step
(6).
830 All three reference implementations enforce both checks, and vectors
831  $\texttt{W-N06}$ -- $\texttt{W-N08}$  of Section~ $\ref{sec:kats}$  pin them.
832  $\end{remark}$ 
same file
1983  $\texttt{W-N06}$ -- $\texttt{W-N08}$  are  $\emph{validly tagged}$  but
structurally
1984 malformed and therefore reach the post-authentication checks of
1985  $\mathsf{Dec}$  step (3): a token count that is not a multiple of
1986  $\mathrm{MAX\_NOISE\_RUN}+1$ ; a real-token count that is such a
multiple but
1987 is not  $B+2$  for any reachable bucket  $B$ ; and a padded length prefix
1988 claiming more codepoints than the padded buffer holds. Each positive
vector

```

The modulo-bias remark names $\gcd(a, k) = 1$ as the one correctness property that depends on the reduced values, but no correctness property depends on them at all, and the property that would matter for invertibility is $a < k$ rather than coprimality

"Remark 3.7, Modulo bias" is rewritten end to end this round and is now, on the whole, right. It retracts the previous version's inverted domination argument in as many words — "We do *not* claim that this bias is 'dominated by' the PRF advantage of HMAC-SHA256; that comparison is not meaningful, since a small PRF advantage says nothing about the uniformity of a reduced output" — substitutes the correct justification, that no result in Section 4 requires any of the reduced values to be uniform because they are computed underneath the keystream mask and never observed in the clear, withdraws the noise threshold's claimed exemption with figures we independently recomputed and confirm (about 4.17 preimages per output, up to +20% relative deviation, total variation distance ≈ 0.033), and closes with a paragraph delimiting the narrow boundary of the argument. It is a model correction and it closes the previous round's finding in full.

One clause in it is wrong. Between the masking justification and the threshold correction, the remark says: "The one correctness property that does depend on them, $\gcd(a, k) = 1$, follows from the range $a \in [1, k - 1]$ and holds for every draw regardless of bias" (lines 554-556). Two things are mistaken here.

First, no correctness property depends on the reduced values. Both parties recompute the addend independently: Enc's emission loop derives a from $(sk, N, real_idx)$ and Dec's step (7) derives the same a from the same three inputs, so the recovery $c = (t - a)/k$ is exact for *any* value a whatever — biased, unbiased, or arbitrary — as long as the two sides agree. "Proposition 3.14, Correctness"'s proof says precisely this and does not mention coprimality: "Enc's token-emission loop and Dec's loop-inversion step both compute the noise-position oracle, addends, and padding codepoints as pure, deterministic functions of $(sk, N, position)$, so with the same sk and N they compute identical intermediate values at every position" (lines 866-874). The remark therefore asserts a dependency that its own correctness proof demonstrates does not exist, and in doing so it leaves a reader with the impression that the addend range is a correctness requirement rather than a free choice.

Second, coprimality is the wrong property even for the argument the remark seems to have in mind. If one imagined recovering c and a from $t = ck + a$ *without* recomputing a — by taking $c = \lfloor t/k \rfloor$ and $a = t \bmod k$ — what that needs is $0 \leq a < k$, which the range $[1, k - 1]$ supplies. $\gcd(a, k) = 1$ is a strictly stronger and irrelevant statement; it happens to be implied here only because k is prime and a is a nonzero residue, which is a coincidence of the parameter choice rather than a requirement. The paper does state $\gcd(token, k) = 1$ separately at lines 586-590, where it is used for nothing either, and the only place the addend's relationship to k does real work is the compromise caveat's trial-division search (lines 2066-2091), which needs the addend to be recomputable from sk alone — again not coprimality.

The consequence is nil for every bound in the paper: "Lemma 4.10, Hiding" is proved for a fixed token blob and is bias-agnostic, and the integrity theorems depend only on the tag. We re-verified both. The severity is low and the finding is confined to this clause. It is worth fixing because the clause sits inside the one remark whose entire purpose is to state precisely what does and does not depend on the reduced values, and be-

cause "the one correctness property that does depend on them" is exactly the sentence a reimplementer will read as a constraint they must preserve.

Recommendation

Delete the clause or invert it. The accurate statement is that **no** property in this paper depends on the reduced values — correctness because both parties recompute them from $(sk, N, \text{position})$ and therefore agree whatever they are, confidentiality because they are masked, integrity because it rests on the tag — which is a cleaner and stronger version of the point the remark is making, and which makes "no rejection sampling is required" follow without exception.

If a note on the addend range is wanted, state what the range actually buys: $a \in [1, k - 1]$ keeps every token in $[k, (c + 1)k - 1]$ and keeps $a \neq 0$, so a token is never a bare multiple of k , and that is a property of the encoding rather than of correctness.

Then either drop the gcd statement at lines 586-590 as well, or say what consumes it.

Client Comment

/docs/napseq-eprint-v4.tex

```

554 clear. The one correctness property that does depend on them,
555  $\gcd(a,k)=1$ , follows from the range  $a \in [1,k-1]$  and holds for
    every
556 draw regardless of bias. No rejection sampling is therefore required
    — not
    same file

586 Real and noise tokens are integers  $c \cdot k + a$  for codepoint  $c$ 
587 (or a noise character), key element  $k = \mathbf{k}[i \bmod K]$ , and
    addend
588  $a$  from Definition~\ref{def:token-prims}. Because  $k$  is prime and
    every
589 addend lies in  $[1,k-1]$ ,  $\gcd(a,k)=1$  and hence
     $\gcd(\text{token},k)=1$ 
590 for every token, real or noise.
    same file

866 Step (7) then unmask with  $\mathbf{ks}(N)$ , the same keystream
867  $\mathbf{Enc}$  masked with (a function of  $\mathbf{sk}$  and the
    recovered
868  $N$  alone), recovering  $B$  and hence the token vector.
     $\mathbf{Enc}$ 's

```

869 token-emission loop and Dec 's loop-inversion step both
 870 compute
 871 the noise-position oracle, addends, and padding codepoints as pure,
 872 deterministic functions of $(\text{sk}, N, \text{position})$, so
 873 with the
 874 same sk and N they compute identical intermediate values
 at
 every position; emission and inversion are therefore exact inverses,
 and
 stripping the length prefix and padding yields M .

same file

2066 `\item[Scope of  $\text{sk}$  and  $\text{k}$  compromise.]`
 sk
 2067 is the sole cryptographic secret in this construction: every
 HMAC
 2068 domain derivation – the noise schedule, the padding, the
 2069 per-position addends, the keystream, and the tag – is keyed by
 2070 sk alone (Section~\ref{sec:domains}).
 Consequently, an
 2071 adversary who compromises sk but not k
 can
 2072 recover any real token's plaintext codepoint directly: the
 2073 per-position addend's raw HMAC output depends only on
 2074 sk , so for a given token the adversary can, for
 each of
 2075 the $\{, \}$ 947 candidate primes in $\mathcal{P}$, compute
 2076 the corresponding addend and check whether the resulting
 quotient
 2077 is an exact integer in a valid codepoint range – an
 2078 $\mathcal{O}(|\mathcal{P}|)$ trial-division search that
 succeeds in
 2079 microseconds. k 's secrecy therefore adds
`\textbf{no}`
 2080 independent confidentiality margin once sk is
 known:
 2081 compromise of sk alone is equivalent to a full
 break,
 2082 regardless of whether k also leaked. Operationally,
 2083 k and sk must therefore be protected
 with
 2084 equal rigor, as a single unit – not because splitting them
 across
 2085 separate storage or trust domains buys compartmentalized
 2086 defense-in-depth (it does not), but because sk

2087 | compromise alone is already fatal. This is consistent with,
 and
 2088 | does not weaken, the security theorems of
 2089 | Section~\ref{sec:security}, which assume the entire key
 material
 2090 | is secret from the adversary, exactly as for any symmetric AEAD
 2091 | scheme (a leaked AES-256 key is likewise a full break of
 AES-GCM).

CVF-9: openCategory: **algorithm**

The proposition establishing that the security theorems survive deleting the arithmetic layer asserts that its proofs use exactly four named facts, and the enumeration is incomplete: the chosen-ciphertext proof additionally invokes correctness, which is never established for the reduced construction

Section 6.1 specifies **NAPQES-L** — the construction of “Definition 3.10” with the token-emission loop replaced by the identity map on padded codepoints, each serialised as three bytes big-endian — and “Proposition 6.1, The theorems are indifferent to the arithmetic layer” states that “Theorem 4.8”, “Theorem 4.12”, “Theorem 4.18” and “Theorem 4.20” “hold for NAPQES-L with identical statements and identical bounds, by the same proofs” (lines 1744-1749). This is the honest comparison the previous round asked for, and it does most of what it claims. Its proof reads: “Each proof uses exactly four facts about the masked payload: that domain separation holds ..., which is unchanged; that the nonce-collision bound holds ...; that the payload is assembled entirely from domains other than $0x07$...; and that the payload length is a function of B alone No proof step examines the token map, the noise schedule, or $\mathbf{k}$ ” (lines 1751-1763).

All four facts do hold for NAPQES-L and we verified each. Domain separation is a property of the input encoding and is untouched by the payload change. The nonce-collision bound depends only on domain $0x0A$. NAPQES-L’s payload is the padded codepoint sequence, built from the plaintext and domain $0x06$ alone and hence disjoint from the domain- $0x07$ keystream, so the one-time-pad step of “Lemma 4.10, Hiding” still applies. And its length is exactly $3(B + 2)$ bytes, a function of the bucket. We also checked the serialisation width the section does not: codepoints are below 2^{21} and the two length-prefix codepoints are below 256, so three bytes suffice throughout.

The enumeration is nonetheless not exhaustive, and the proposition asserts that it is. The proof of “Theorem 4.20, IND-CCA-det” — one of the four theorems in scope — takes a game hop from H_0 to H_1 in which decryption queries are answered from a table, and justifies the agreement of the two games on table hits by appealing to correctness: “On every query with $(c, A) \in T$ the two games agree: H_1 returns the recorded plaintext, and H_0 returns the same value by correctness (Proposition 3.14)” (lines 1541-1544). That is a fifth fact, it is not about the masked payload, and “Proposition 3.14” is stated and proved for Π — “the triple Π ... and only this scheme” — not for NAPQES-L. NAPQES-L’s

correctness is obvious, the token map being the identity and the inversion a three-byte deserialisation, but it is nowhere stated and nowhere proved, so as the paper stands the chosen-ciphertext quarter of "Proposition 6.1" rests on an unproved premise about a construction whose correctness is never asserted.

This is a completeness defect in an enumeration, not an error: three of the four theorems transfer exactly as claimed, the fourth almost certainly does too, and the missing step is a sentence. The severity is low. It matters because "Proposition 6.1" is cited from four places as establishing that removing the arithmetic layer "changes no theorem in this paper" — Section 6's "not claimed" list (lines 1706-1709), the proposition itself, the NAPQES/NAPQES-L comparison table's first row (line 1771), and the conclusion (lines 2164-2166) — and because "exactly four facts" is a strong claim of the kind a reader checks. It is also the claim that carries the paper's most consequential architectural concession, so it should be airtight.

Recommendation

Either add correctness to the enumeration and discharge it, or drop the word "exactly".

The discharge is short: state that NAPQES-L is correct, by the same argument as "Proposition 3.14" with the token-emission and inversion steps replaced by a three-byte serialisation and its inverse, and note that 3 bytes suffice for every codepoint in $\mathcal{M}$ and for the two length-prefix codepoints. Then list correctness as the fifth fact, noting that it is what "Theorem 4.20"'s first game hop consumes.

While the enumeration is being revised, it is worth saying explicitly that the four facts are facts about the *payload* and that "Theorem 4.20" additionally depends on the scheme's correctness, since that is the distinction the current phrasing elides.

Client Comment

/docs/napseq-eprint-v4.tex

```

834 \begin{proposition}[Correctness]\label{prop:correctness}
835 For every  $\lambda$ , every
836  $(\mathbf{k}, \mathsf{sk}) \xleftarrow{\mathsf{KeyGen}}(1^\lambda)$ ,
837 every  $A \in \mathcal{A}$ ,  $M \in \mathcal{M}$ :
838  $\mathsf{Dec}(\mathbf{k}, \mathsf{sk}, A, \mathsf{Enc}(\mathbf{k}, \mathsf{sf}(M))) = M$ .
839 \end{proposition}
840 \end{proposition}
841
842 \begin{proof}
843 Let  $C = N || \widetilde{B} || T$  be the output of

```

844 $\text{Enc}(\mathbf{k}, \text{sk}, A, M)$. We first check that the
 845 two tag
 846 inputs coincide, rather than assuming tag agreement.

847 By Lemma~\ref{lem:length-det},
 $|C| = 16 + 8R(\text{MAX_NOISE_RUN} + 1) + 32$
 848 with $R = B(|M|) + 2 \geq 1$, so $|C| \geq 2928 > 48$ and
 849 $N_{\text{tok}} = (|C| - 48) / 8 = R(\text{MAX_NOISE_RUN} + 1)$ is a
 850 non-negative integer divisible by $\text{MAX_NOISE_RUN} + 1$: steps
 851 (1)
 852 and (3) of Dec do not reject, and step (3) recovers
 853 exactly the
 854 R used by Enc . Step (2) parses the leading 16 bytes as
 855 N and
 856 the trailing 32 as $T' = T$, leaving the middle bytes as exactly the
 857 $\widetilde{B}$ that Enc emitted; in particular N is
 858 $\text{read from the ciphertext}$, not re-derived, so no circularity
 859 with the
 860 synthetic-nonce derivation arises. Hence Dec step (4)
 861 evaluates
 862 $\text{Derive}_{\{0x03\}}$ on $\text{TagIn}(A, \widetilde{B})$ with
 863 the same
 864 sk , the same N , the same caller-supplied A , and the
 865 same
 866 $\widetilde{B}$ as Enc step (6) did
 867 (Definition~\ref{def:taginput}). The two tag inputs are
 868 byte-identical, so
 869 $T = T'$ and step (5) does not reject. Since $R = B(|M|) + 2$ with
 870 $B(|M|) \in \{2^4, \dots, 2^{16}\}$, step (6) does not reject either,
 871 and since
 872 the length prefix written by Enc satisfies $n = |M| \leq B(|M|) = R - 2$,
 873 neither does step (8).

874 Step (7) then unmask with $\text{ks}(N)$, the same keystream
 875 Enc masked with (a function of sk and the
 876 recovered
 877 N alone), recovering B and hence the token vector.
 878 Enc 's
 879 token-emission loop and Dec 's loop-inversion step both
 880 compute
 881 the noise-position oracle, addends, and padding codepoints as pure,
 882 deterministic functions of $(\text{sk}, N, \text{position})$, so
 883 with the
 884 same sk and N they compute identical intermediate values
 885 at

873 every position; emission and inversion are therefore exact inverses,
 874 and
 875 stripping the length prefix and padding yields M .
 $\text{\textbackslash end\{proof\}}$

same file

1539 $\text{\textbackslash paragraph\{Transition.\}}$ Let D be the event that $\mathcal{A}$
 submits a
 1540 decryption query (A, c) with $(c, A) \notin T$ for which
 1541 $\text{\textbackslash mathsf\{Dec\}(\text{\textbackslash mathbf\{k\}, \text{\textbackslash mathsf\{sk\}, } A, c) \neq \text{\textbackslash bot\}}$. On every query
 with
 1542 $(c, A) \in T$ the two games agree: H_1 returns the recorded
 plaintext, and
 1543 H_0 returns the same value by correctness
 1544 (Proposition~ $\text{\textbackslash ref\{prop:correctness\}}$). On every query with
 $(c, A) \notin T$
 1545 the two games agree unless D occurs. Hence H_0 and H_1 are
 identical
 1546 until D , and $|\Pr[H_0 = 1] - \Pr[H_1 = 1]| \leq \Pr[D]$.

same file

1706 layer contributes nothing to Theorem~ $\text{\textbackslash ref\{thm:ind-cpa\}}$,
 1707 Theorem~ $\text{\textbackslash ref\{thm:int-ctxt\}}$, or Theorem~ $\text{\textbackslash ref\{thm:ind-cca\}}$, and
 1708 removing it changes none of their statements or bounds
 1709 (Proposition~ $\text{\textbackslash ref\{prop:lean-theorems\}}$).

same file

1744 $\text{\textbackslash begin\{proposition\}}$ [The theorems are indifferent to the arithmetic
 layer]
 1745 $\text{\textbackslash label\{prop:lean-theorems\}}$
 1746 Theorem~ $\text{\textbackslash ref\{thm:ind-cpa\}}$, Theorem~ $\text{\textbackslash ref\{thm:lh-ind-cpa\}}$,
 1747 Theorem~ $\text{\textbackslash ref\{thm:int-ctxt\}}$ and Theorem~ $\text{\textbackslash ref\{thm:ind-cca\}}$ hold for
 NAPQES-L
 1748 with identical statements and identical bounds, by the same proofs.
 1749 $\text{\textbackslash end\{proposition\}}$
 1750
 1751 $\text{\textbackslash begin\{proof\}}$
 1752 Each proof uses exactly four facts about the masked payload: that
 domain
 1753 separation holds (Lemma~ $\text{\textbackslash ref\{lem:domsep\}}$), which is unchanged; that
 the
 1754 nonce-collision bound holds (Lemma~ $\text{\textbackslash ref\{lem:nonce-collision\}}$), which
 1755 depends only on domain $\text{\textbackslash texttt\{0x0A\}}$ and is unchanged; that the
 payload
 1756 is assembled entirely from domains other than $\text{\textbackslash texttt\{0x07\}}$, so
 that the
 1757 one-time-pad step of Lemma~ $\text{\textbackslash ref\{lem:hiding\}}$ applies – in NAPQES-L the

```

1758 | payload is the padded codepoint sequence, built from the plaintext and
1759 | domain  $\texttt{\$}\texttt{\texttt{\texttt{0x06}}}\texttt{\$}$  alone, so this still holds; and that the
      | payload
1760 | length is a function of  $\texttt{\$B}\texttt{\$}$  alone, which for NAPQES-L is immediate,
      | since
1761 | it is exactly  $\texttt{\$3(B+2)}\texttt{\$}$  bytes. No proof step examines the token map,
      | the
1762 | noise schedule, or  $\texttt{\$\mathbf{k}}\texttt{\$}$ .
1763 |  $\texttt{\backslash end\{proof\}}$ 
      |
      | same file
1771 | IND-CPA-det, INT-CTXT, IND-CCA-det & identical bounds & identical
      | bounds  $\texttt{\backslash\backslash}$ 
      |
      | same file
2164 | nothing to it (Proposition~ $\texttt{\backslash ref\{prop:expansion-neutral\}}$ ), and
      | removing that
2165 | layer changes no theorem in this paper
2166 | (Proposition~ $\texttt{\backslash ref\{prop:lean-theorems\}}$ ). Separately from the proofs,
      | the

```

CVF-10: openCategory: **procedural**

The contributions promise that each of the three evidence sections states what its evidence does and does not establish; two of the three do not, and the conclusion describes the statistical suite as having been run on the keystream where the section itself reports it was run on ciphertext

The previous round found the statistical, known-answer and fuzz results presented as validation of the security claims, and the core of that is now fixed, thoroughly. The abstract reports all three and then scopes them: "evidence for implementation agreement and decoder robustness respectively, not for any security claim, which rests on the theorems alone" (lines 96-101). The conclusion does the same at length: "these are evidence of implementation agreement, statistical quality and decoder robustness respectively, and none of them is evidence for a security claim, all of which rest on the theorems alone" (lines 2170-2173). Section 10 states it for the corpora in its own closing sentence: "these are evidence of cross-implementation agreement only; no security claim in this paper rests on them" (lines 2005-2009). The word "validated" is gone from the conclusion. That is the substance of the finding closed.

Contribution 8 makes a specific promise about the sections themselves, however: "We report NIST SP 800-22 Rev. 1a statistical test results on ciphertext output (Section 9), the cross-implementation KAT corpora (Section 10), and a fuzz harness covering the full decoder pipeline (Section 11), **and state in each case what the evidence does and does not establish**" (lines 181-186). Section 10 does. Sections 9 and 11 do not.

Section 9 (lines 1956-1966) is eleven lines reporting that the full suite was applied to 10^7 bits of ciphertext and that all fifteen eligible tests passed at $\alpha = 0.01$, with a pointer to the reproduction script and no statement of what a pass means — which for this construction is close to nothing, since the tested bits are a pseudorandom keystream exclusive-ored with structured data, so any failure would be a break of HMAC-SHA256 rather than a finding about NAPQES, and the suite would pass equally on the keystream alone. Section 11 (lines 2012-2023) reports two harnesses, the entry points they target and that no panics were observed over 10^6 corpus entries, again with no statement of scope. A reader who reads the sections rather than the abstract gets the results without the framing, and the framing is the whole point of the repair.

The conclusion also misdescribes the object of one test. It says “the keystream passes the NIST SP 800-22 Rev. 1a suite (Section 9)” (lines 2168-2169), while Section 9 says the suite was applied to “ 10^7 bits of NAPQES ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key” (lines 1961-1963), and contribution 8 says “statistical test results on ciphertext output”. Ciphertext and keystream are different objects, and the difference is exactly the one that determines what the test could have shown: a suite run on the keystream tests the derivation, while a suite run on ciphertext tests the derivation composed with the masking, and since the masked blob is uniform whenever the keystream is, neither result distinguishes a construction that hides the plaintext from one that does not. Three sites name three subjects, and a reader cannot tell which was measured.

Severity is low; nothing in the security argument depends on any of it. It is worth fixing because contribution 8 makes a checkable promise that two sections do not keep, and because the correction is two sentences and one word.

Recommendation

Add the scoping sentence to Sections 9 and 11 in the form Section 10 already uses.

For the statistical suite, say that a pass establishes that the masked output shows no gross statistical structure, that this is a sanity check on the keystream derivation and the masking step rather than evidence for any security claim, that a failure would indicate a defect in the implementation or a break of the underlying primitive rather than a property of the mode, and that NIST positions SP 800-22 as a tool for evaluating random and pseudorandom generators.

For the fuzz harnesses, say that they establish decoder robustness against malformed input over the reported corpus and nothing about confidentiality or integrity.

Then reconcile the subject: state in one place whether the suite was run on ciphertext, on the keystream, or on both, and use that same word in Section 9, contribution 8 and the conclusion.

Client Comment

/docs/napseq-eprint-v4.tex

```

181 | \item We report NIST SP 800-22 Rev.\ 1a statistical test results on
182 | ciphertext output (Section~\ref{sec:sts}), the
cross-implementation
183 | KAT corpora
184 | (Section~\ref{sec:kats}), and a fuzz harness covering the full
185 | decoder pipeline (Section~\ref{sec:fuzz}), and state in each
case
186 | what the evidence does and does not establish.

```

same file

```

1956 | \section{NIST SP 800-22 Statistical Analysis}
1957 | \label{sec:sts}
1958 | %

```

```

1959 |
1960 | We applied the full NIST SP 800-22 Rev.\ 1a Statistical Test
1961 | Suite~\cite{sp800-22} to  $10^7$  bits of NAPQES ciphertext output
generated
1962 | by encrypting a cycling corpus of printable-ASCII plaintext with a
1963 | 10-element prime key. All 15 eligible SP 800-22 tests passed at the
1964 |  $\alpha=0.01$  significance level. Results are reproducible using
1965 | \texttt{tests/sts\_pipeline.py} in the reference implementation
1966 | repository.

```

same file

```

2005 | Both corpora, together with the regression suites across the Python,
Rust
2006 | and C implementations, also check Proposition~\ref{prop:correctness}
2007 | empirically. As stated in Section~\ref{sec:intro}, these are evidence
of
2008 | cross-implementation agreement only; no security claim in this paper
rests
2009 | on them.

```

same file

```

2012 | \section{Fuzz Testing}
2013 | \label{sec:fuzz}
2014 | %

```

```

2015 |
2016 | Two fuzz harnesses are provided. The Python harness
2017 | (\texttt{tests/fuzz\_atheris.py}, using \texttt{atheris}) targets
2018 | \texttt{decrypt\_bytes\_v8} and the fixed-width token decoder,
verifying
2019 | that no exception other than \texttt{ValueError} escapes the public
API.

```

```

2020 | The Rust harness (\texttt{rust/fuzz/fuzz\_targets/decode\_bytes.rs},
      | using
2021 | \texttt{cargo-fuzz}) targets the Rust \texttt{decrypt\_bytes\_v8}
      | entry
2022 | point with multiple key sizes and AAD values. No panics were observed
2023 | across  $10^6$  corpus entries in either harness.
      |
      | same file
2168 | corpora of Section~\ref{sec:kats}, the keystream passes the NIST SP
      | 800-22
2169 | Rev.~1a suite (Section~\ref{sec:sts}), and the decoders survive the
      | fuzz

```

6. Conclusion

Version 4 is the strongest revision of this document we have reviewed, and the improvement is structural rather than cosmetic. The single high-severity defect of the previous round — a decryption algorithm that recomputed the authentication tag over a four-byte associated-data length prefix where encryption used eight, so that no honest ciphertext verified and the stated correctness property was false — is repaired at the root: the tag input is now a single named quantity to which every occurrence of the tag in the document refers, and we confirmed mechanically that no four-byte form survives anywhere in the source. The same edit makes the ciphertext-integrity proof match the scheme it is stated about, which it previously did not.

Three further repairs are worth recording because each closes a class of defect rather than an instance. The domain-separation lemma is restated over its entire argument domain, including tuples chosen adversarially, which is the scope the integrity proof needs and had been reaching beyond. All four security notions are now formally defined, where the previous version defined one and asserted three; the chosen-ciphertext result in particular is no longer a proof sketch resting on a composition theorem that does not cover deterministic encryption, but a complete argument given in place. And the architectural question this engagement has raised since its first round — what the noise-injection and prime-token layer actually buys, given that it accounts for essentially the whole cost of the construction — is now answered directly, in a section that separates what is proved from what is merely argued, and in a specification of the same construction with that layer removed together with a proof that no theorem changes. That is an unusually honest thing for a paper to do about its own design, and the paper deserves credit for it.

We reduced all fifteen proofs in the document to their inference skeletons and checked

every step independently against what it cites or computes. None contains an unsupported step, a circular step or a false step. We also recomputed every numeric claim in the document — the prime-set count by independent sieve, the min-entropy and key-space figures, the post-quantum figures, the maximum token value and its margin against the eight-byte field, the minimum ciphertext length, the noise threshold's deviation from uniform, and every digit of the measured length-leakage table — and all of them agree. The rounding-and-approximation defects that produced findings in each of the previous three rounds are gone.

Ten findings remain, none of them high or critical. Three are moderate. The first concerns the new length-hiding result: the notion, the theorem and the quantified-leakage corollary are all sound, and we verified them, but the notion is parameterised by the paper's own padding profile, so the zero-query separation exhibited against AES-GCM and ChaCha20-Poly1305 measures the presence of that profile rather than a property of the schemes compared, and any AEAD scheme achieves the same notion at the same bound by applying the same profile before encrypting. The paper's own analysis contains everything needed to say so and stops one step short. The second concerns the one benefit still claimed for the arithmetic layer, extra adversary work after a nonce collision, which is asserted without a bound while the paper's own compromise caveat prices the identical search over the same public prime set at an operation count that succeeds, in its own words, in microseconds; a deployment is thereby invited to choose a fifty-three-fold bandwidth cost against a margin worth roughly nineteen bits. The third concerns an implementation divergence the paper documents but does not resolve: the C port maps each input byte to one codepoint where the specification and the other two ports use Unicode scalar values, so a ciphertext produced by one port and decrypted by another passes every check and returns a different plaintext, silently, and the known-answer corpus offered as the evidence of cross-implementation agreement is confined to the ASCII domain in which the divergence cannot appear.

The seven minor findings are residues and precision defects rather than design problems. Four of them are single-site remainders of repairs that are otherwise complete — a comparison-table cell still quoting expansion in the wrong unit, a front-matter formula still missing its truncation, a decryption step still naming a raw digest where it means a reduced value, and two evidence sections still omitting the scoping statement the contributions promise for all three. One records that a recommendation to adopt the zero-leakage padding profile omits the message-space restriction and the bandwidth cost that profile carries, and that the profile is neither transmitted nor detectable, so a receiver cannot enforce the guarantee it is told to rely on. One records a clause naming a correctness dependency that the document's own correctness proof shows does not exist. One records that the proposition establishing the theorems' indifference to the arithmetic layer asserts an exhaustive list of the facts its proofs use, and the list omits correctness, which the chosen-ciphertext proof invokes and which is not established for the reduced construction.

Two items remain outside what a paper-only round can settle, for the third consecutive round. The document asserts that all three reference implementations enforce the two decoder checks it newly specifies, and names test vectors that pin them; the specification half of that is now correct and complete, but the implementation half cannot be verified without the code and the corpus. And the unauthenticated-streaming caveat that the third round dropped along with the streaming format it described is neither re-

stated nor contradicted here, while the caveats section has meanwhile grown to cover implementation-specific limitations that are not properties of the scheme — so the place such a caveat would belong now demonstrably exists. Both are carried forward, and we recommend that the next submission include the reference distribution so that they can be closed.

Our overall assessment is that the security argument for the construction is now in good order: the notions are defined, the proofs are sound as far as we can check them, the bounds are the right bounds for the games as stated, and the assumptions are standard and clearly located. What remains open is not the cryptography but the presentation of the comparative claims built on top of it, and one implementation that does not implement the scheme the paper specifies.

Client Comment



About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

Email

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

[@ABDKConsulting](https://twitter.com/ABDKConsulting)

LinkedIn

company/abdk-consulting